OUTCOMESERVE: CRASH-CONSISTENT RECOVERY FOR LONG-HORIZON AGENTIC LLM SERVING

Shirley Li [1] **Lordphone Wen** [1] **Victoria Zhu** [1]

1 PROBLEM AND MOTIVATION

LLM agents alternate between model generation and tool calls. A worker crash can separate what an agent remembers from what its tools have already done: an order may be created just before the worker records the result. Retrying can create a duplicate order; restarting repeats completed inference and may repeat earlier side effects.

Research question. Can an agent-serving runtime resume after a worker crash while preserving committed tool effects and reducing repeated model computation? We propose **OutcomeServe**, a vLLM-based runtime that reconciles tool outcomes with a durable conversation log, then rebuilds inference state from the recovered token prefix. The challenges are resolving uncertain tool completion, aligning tool results with model context, and balancing durable logging overhead against recovery cost as context grows.

2 RELATED WORK AND AVAILABLE ARTIFACTS

AgentRewind restores aligned context and workspace checkpoints to recover from unsuccessful trajectories (Zhuang et al., 2026); **Crab** selects filesystem checkpoint granularity for agent sandboxes (Wu et al., 2026). Both have public code, but their controlled-environment recovery does not resolve externally committed tool effects. **Belayer** coordinates rollout-worker and environment recovery for agentic RL, including rebuilding KV contents from logged token prefixes (Zhou et al., 2026). Its paper describes a Slime implementation; a standalone release was not identified in the checked paper. **DéjàVu** provides open-source KV-cache streaming and replication for inference recovery (Strati et al., 2024), without coordinating agent-level tool commits.

Durable workflow frameworks such as open-source **LangGraph** already support checkpointed agent execution (LangChain, 2026). Our contribution is a scoped integration and evaluation of tool-outcome reconciliation with LLM re-prefill, rather than a claim to invent durable execution. We will use open-source **vLLM** and the Apache-2.0 **Qwen3-4B** model (vLLM Project, 2026; Qwen Team, 2025); artifact links are included in the references.

3 PROPOSED APPROACH AND SCOPE

Failure model. We target fail-stop crashes of the agent runner or inference process. The durable log and tool service survive, and only one runner owns a task at a time. Tools execute sequentially. For supported writes, the tool service must atomically commit the side effect and its result under a stable operation ID, and return the stored result on duplicate submissions. We will implement this contract using a transactional database. An external API without this contract cannot receive the same guarantee; an ambiguous outcome must be surfaced rather than blindly retried.

Commit and recovery protocol. The runtime combines an agent runner, vLLM, a durable log, a transactional tool service, and a fault injector:

1. **Prepare.** Before dispatch, persist the generated tool call, arguments, and operation ID derived from the task and step. Incomplete, uncommitted generation may be discarded after a crash.
2. **Execute or reconcile.** Submit the operation ID to the tool service. If the acknowledgment is lost, resubmit the same ID: the service returns the committed result or atomically executes the operation once. A local `PREPARED` record alone does not prove non-execution.
3. **Commit context.** Atomically append the tool result and advance the committed prefix boundary. Recovery reuses recorded outputs and results, then submits the restored token prefix to vLLM for re-prefill, rebuilding the attention KV cache before continuing.

The intended invariant is that every completed step appears once in the committed transcript and matches one tool-side outcome; a pending committed effect is reconciled before further generation. The log stores token IDs and model versions. This preserves committed history without requiring

[1]. Correspondence to: Shirley Li <shirley3@andrew.cmu.edu>, Lordphone Wen <lordphow@andrew.cmu.edu>, Victoria Zhu <qiyuez@andrew.cmu.edu>.

11-868 Large Language Model Systems Project Proposal.

future model outputs to be identical. GPU-cache replication, sandbox rollback, and storage failures are outside scope. If time permits, we will vary prefix snapshot intervals while retaining every tool-intent record to study logging versus replay cost.

4 Evaluation Plan

Workloads and faults. We will build order tasks with 5, 10, and 20 tool steps, mixing reads, idempotent writes, and non-idempotent actions such as inventory decrements. Deterministic tool services and final-state verifiers make side effects auditable. Scripted model traces isolate protocol correctness; real Qwen3-4B runs measure serving performance. We will vary prefixes over 2K, 8K, and 16K tokens and inject crashes during generation, before dispatch, after tool commit but before result logging, and after result logging. CPU tests will also crash the recovery path itself.

Comparisons. We compare full-task restart, naive step retry, transcript checkpointing without tool reconciliation, and OutcomeServe, plus failure-free runs with and without logging. All use the same model, tool implementation, and worker-start policy; the naive variants omit stable-ID deduplication. External state persists across a crash. A separate resettable-environment restart experiment isolates recomputation cost, with reset cost reported. This avoids treating external rollback as free or generally available.

Metrics and hypotheses. Correctness measures are duplicate effects, tool-state mismatches, and task success. We target zero observed consistency violations for supported tools, with task errors reported separately. Performance measures are p95 crash-to-next-token latency, end-to-end task latency, re-prefilled input tokens, regenerated output tokens, and failure-free latency overhead (target: below 5%). Recovery time will be split into worker startup, reconciliation, and re-prefill. We hypothesize that resume saves completed decoding and tool work, with greater savings at longer task horizons; long prefixes may reduce this benefit. Paired task seeds, at least 20 repetitions per selected condition, and confidence intervals will distinguish consistent gains from noise. A reconciliation ablation tests the correctness mechanism directly.

Resume-versus-restart crossover analysis. Beyond verifying recovery correctness, we ask a second systems question: *At what prefix length and completed-task fraction does re-prefill-based resume stop outperforming full-task restart?* We decompose the measured recovery costs as

$$T_{\text{resume}} = T_{\text{startup}} + T_{\text{reconcile}} + T_{\text{prefill}}(L),$$

and

$$T_{\text{restart}} = T_{\text{startup}} + T_{\text{regenerate}}(k) + T_{\text{tools}}(k) + T_{\text{reset}},$$

where L is the committed prefix length and k is the number of completed tool steps. T_{reset} applies only to the resettable-environment baseline and will be reported separately rather than treating external rollback as free. We will vary L and k, then report a heatmap of $\Delta T(L, k) = T_{\text{restart}} - T_{\text{resume}}$, where a positive value indicates that resume is faster. This analysis will identify the crossover region in which re-prefill cost outweighs the completed generation and tool work preserved by recovery. In addition to crash-to-next-token latency, we will measure re-prefill throughput and GPU utilization during recovery.

5 Team and Milestones

Team Shirley Li will coordinate the project and develop the agent runtime, crash-consistency protocol, vLLM integration, and recovery mechanism; Lordphone Wen will manage GPU/CPU infrastructure, worker lifecycle, fault injection, hardware monitoring, and performance benchmarks; Victoria Zhu will implement durable logging, parse execution traces, reconstruct trajectories, validate consistency, and aggregate metrics. All members share integration, experiments, and writing.

5.1 Eight-week plan

Table 1. Planned eight-week schedule.

Week	Milestone
1	Finalize failure model and interfaces
2	Agent workload, tools, and verifier
3	vLLM integration and fault injection
4	Durable log and state machine
5	Recovery and baseline implementations
6	Correctness experiments and debugging
7	Performance experiments and ablations
8	Analysis, demo, report, and presentation

The minimum viable result compares full restart, naive retry, and crash-consistent resume across the specified failure boundaries.

6 Infrastructure and Compute Budget

We request one L40S 48 GB or A100 40 GB GPU (or equivalent), 16–32 CPU cores, 64 GB RAM, and 200 GB NVMe storage. A second GPU enables a warm replacement replica; one GPU suffices for process-restart experiments. Qwen3-4B requires roughly 8 GB for BF16 weights before runtime overhead, with a 24 GB GPU as a fallback. We budget 90–120 GPU-hours: 20–30 for integration, 10–20 for pilots, 40–50 for evaluation, and 20 for repeated runs. With two reserved GPUs, this corresponds to 45–60 wall-clock hours. CPU-only protocol tests will reduce GPU use.

REFERENCES

LangChain. LangGraph: Persistence, 2026. URL https://docs.langchain.com/oss/python/langgraph/persistence. Code: https://github.com/langchain-ai/langgraph. Accessed September 29, 2026.

Qwen Team. Qwen3-4B model card, 2025. URL https://huggingface.co/Qwen/Qwen3-4B. Open weights, Apache-2.0 license. Accessed September 29, 2026.

Strati, F., McAllister, S., Phanishayee, A., Tarnawski, J., and Klimovic, A. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. *arXiv preprint arXiv:2403.01876*, 2024. URL https://arxiv.org/abs/2403.01876. Code: https://github.com/msr-fiddle/dejavu.

vLLM Project. vLLM: A high-throughput and memory-efficient inference and serving engine, 2026. URL https://github.com/vllm-project/vllm. Open-source software. Accessed September 29, 2026.

Wu, T., Chang, C., Cao, L., Gao, W., and Wang, W. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes. *arXiv preprint arXiv:2604.28138*, 2026. URL https://arxiv.org/abs/2604.28138. Code: https://github.com/open-agent-infra/crab.

Zhou, J., Hu, Q., Sun, P., Zhang, X., and Zhang, W. Belayer: Efficient fault tolerance for LLM agentic RL training. *arXiv preprint arXiv:2608.14635v2*, 2026. URL https://arxiv.org/abs/2608.14635v2.

Zhuang, Y., Chen, K., Duan, Y., Zheng, S., Li, J., and Zhang, X.-Y. AgentRewind: Recoverable execution for long-horizon LLM agents. *arXiv preprint arXiv:2608.14380*, 2026. URL https://arxiv.org/abs/2608.14380. Code: https://github.com/Futuresis/replay-agent-recorder.